`$ node breakfast.js --mode=async`

JavaScript [illegible]

// [illegible] Promise [illegible] async/await

sync	promise	await	event loop
[illegible]	[illegible]	[illegible]	[illegible]

`$ cat menu.md`

□□□□

01 / □□□□
□□□□ vs □□□□□

02 / Promise
□□□□□then/catch□□□□□

03 / async & await
□□□□□□ vs □□

04 / Event Loop
VIP □□□□□□□□□

□□□□□□□□□ async □□□□□□□□□□□□□□

PART 01

[illegible]

```javascript
function makeCoffee() {
    console.log("[illegible]...");
    const start = Date.now();
    while (Date.now() - start < 2000) {} // [illegible]
    console.log("[illegible]");
}

function prepareBreakfastSync() {
    makeCoffee(); // 2s
    makeToast();  // 1.5s
    fryEgg();     // 1s
}
```

CONSOLE — 4.5s

[illegible]...
[illegible]
[illegible]...
[illegible]
[illegible]

□□□□□□□□□□

```javascript
function makeCoffee() {
    return new Promise((resolve) => {
        console.log("□□□□□...");
        setTimeout(() => {
            resolve("□□□□□");
        }, 2000); // □□□□□□
    });
}

async function prepareBreakfastAsync() {
    const coffee = await makeCoffee();
    console.log(coffee);
    // ...□□□□
}
```

CONSOLE — ~2s

```
□□□□□□...
□□□□□□...
□□□□□...
□□□□□□□□□□□□□□□□
□□□□□□□□□□□
```

□□□□□

`$ node breakfast.js`

□□□□□□□□□□

□□ vs □□□□□□□□

▶ □□□□□□□□□□□□□□ ▶ □□□□□□□□□□□□

☕ □□□□2s□

🍞 □□□□1.5s□

🍳 □□□1s□

□□□□□ = 2 + 1.5 + 1 = **4.5** □□ □□□□□□ = max(2, 1.5, 1) = **2** □

PART 02

Promise

□□□□□□□□□

pending

□□□□□□

fulfilled

□□□□resolve("□□")

rejected

□□□□reject("□□")

```
let myPromise = new Promise((resolve, reject) => {
    let success = true;
    setTimeout(() => {
        if (success) resolve("□□□");
        else reject("□□□");
    }, 2000);
});

myPromise.then(result => console.log(result))
         .catch(error => console.log(error));
```

□□□□□Promise.race + AbortController

```
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

const fetchWithTimeout = (url, { timeout = 3000 } = {}) => {
    const controller = new AbortController();
    const timer = delay(timeout).then(() => controller.abort());

    const request = fetch(url, { signal: controller.signal });

    return Promise.race([request, timer]).catch((error) => {
        if (error.name === "AbortError") return Promise.reject("□□□□");
        return Promise.reject(error.message || "□□□□");
    });
};
```

Promise.race

□□□□□□□ —— □□□□□

AbortController

□□□□□□□□□□□□□□

PART 03

async / await

then □ → await

```javascript
async function loadSpells() {
    try {
        const res = await fetch("https://hp-api.onrender.com/api/spells");
        const data = await res.json();
        console.log("spells", data.length);
    } catch (err) {
        console.error(err.message);
    }
}
```

□□□□□□□await □□□ then □□□□□□□□□□□□□□ □□□□ async/await□□□□□□ then□

□□□□□□□□□

```javascript
async function serial() {
    const coffee = await makeCoffee();
    const toast = await makeToast();
    return [coffee, toast];
}
```

TOTAL TIME

≈ □□□□□□□□□2 + 1.5 = 3.5s□

□□

□□ token □□ API —— □□□□□□□

□□□□□□□□□

```javascript
async function parallel() {
    const [coffee, toast] = await Promise.all([
        makeCoffee(),
        makeToast(),
    ]);
    return [coffee, toast];
}
```

TOTAL TIME

≈ □□□□□max(2, 1.5) = 2s□

□□

□□□□□□□□□ API

Promise.all vs allSettled

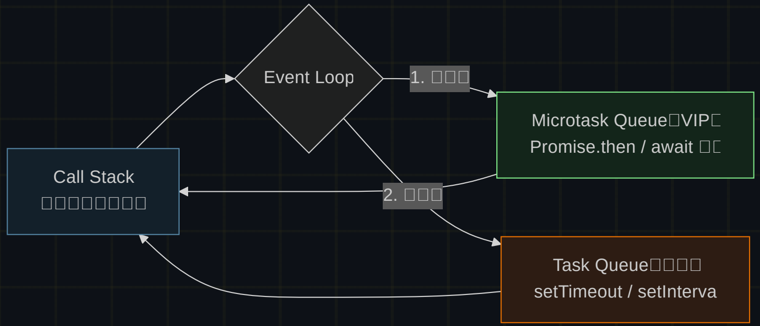
	`Promise.all`	`Promise.allSettled`
[illegible]	[illegible] reject[illegible]	[illegible]
[illegible]	[illegible]	`{status, value/reason}` [illegible]
[illegible]	[illegible]	[illegible]

```
$ [illegible]all [illegible]allSettled [illegible]
```

PART 04

Event Loop

[illegible] VIP [illegible]


Event Loop
1.
Microtask Queue VIP
Promise.then / await
Call Stack
2.
Task Queue
setTimeout / setInterva

VIP

[illegible]

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

ANSWER

1 → 4[illegible]Call Stack [illegible]

→ 3[illegible] VIP [illegible]

→ 2[illegible] 0[illegible]

[illegible]1 → 4 → 3 → 2

[illegible]

[illegible]

[illegible]

Promise

[illegible]pending → fulfilled / rejected

[illegible] vs [illegible]

[illegible] await [illegible] Promise.all

Event Loop

[illegible]

[illegible] = [illegible]

Breakfast served.

□□□□fetch □ AbortController □□□□□□

```
$ node breakfast.js --mode=async
promise → await → race → event loop
```